

UNIVERSIDAD DE SANTANDER — UDES
Ingeniería de Sistemas · Aplicaciones Móviles · 2026

Proyecto Integrador — Unidad 12

Sabores de Colombia

Autores

Omar Felipe Rincon Fonseca — 02210131021

Johan Steven Carreño Daza — 02230131066

Repositorio: <https://github.com/Fekazo/SaboresDeColombia>

Cúcuta, Colombia · Mayo 2026

1. Definición del Problema

La gastronomía colombiana es rica y diversa, reflejando la historia, la cultura y las tradiciones de cada una de sus regiones. Cada departamento cuenta con platos típicos, ingredientes autóctonos y técnicas culinarias propias que forman parte de la identidad nacional. Sin embargo, a pesar de esta gran riqueza cultural, gran parte de este conocimiento gastronómico se encuentra escasamente documentado en formatos digitales accesibles, organizados y fáciles de consultar.

Las personas interesadas en aprender a cocinar platos tradicionales colombianos suelen enfrentarse a la necesidad de buscar información en múltiples fuentes dispersas, como blogs, videos, redes sociales, páginas web y libros digitales. Muchas de estas fuentes presentan información incompleta, desactualizada o, en algunos casos, disponible principalmente en inglés, lo que dificulta el acceso para usuarios hispanohablantes.

Además, existe una limitada organización del contenido basada en criterios geográficos o culturales, lo que complica identificar fácilmente recetas representativas de cada región del país. A esto se suma la falta de recursos visuales de calidad, instrucciones claras paso a paso y datos complementarios sobre ingredientes, preparación y contexto cultural de los platos. Esta situación genera barreras para estudiantes, aficionados a la cocina, turistas y personas interesadas en preservar y aprender sobre el patrimonio culinario colombiano mediante herramientas digitales centralizadas y accesibles.

1.1 Usuarios objetivo

Se identifican dos perfiles de usuario principales:

- Colombiano en el extranjero (ej. Valentina, 28 años, profesional viviendo en el exterior): extraña la comida colombiana y quiere prepararla en casa. Su frustración principal es que las recetas que encuentra en internet están en inglés y no son auténticas.
- Turista gastronómico (ej. Matteo, 34 años, chef aficionado extranjero visitando Colombia): quiere explorar la gastronomía regional pero no conoce los nombres ni los ingredientes locales. Prefiere apps con fotos y guías visuales.

1.2 Requisitos funcionales

ID	Requisito	Pantalla
RF-01	Mostrar un splash de bienvenida con animación durante el inicio.	Splash
RF-02	Presentar tarjetas de regiones geográficas colombianas navegables.	Home
RF-03	Cargar recetas populares aleatoriamente desde TheMealDB al iniciar.	Home
RF-04	Permitir búsqueda de recetas por nombre con resultados en tiempo real.	Búsqueda
RF-05	Listar recetas colombianas al seleccionar una región.	Lista por región
RF-06	Traducir automáticamente el contenido al español via Google Translate API.	Global

ID	Requisito	Pantalla
RF-07	Permitir guardar recetas como favoritos con persistencia local en Room.	Favoritos
RF-08	Permitir eliminar recetas de favoritos deslizando la tarjeta.	Favoritos
RF-09	Mostrar información nutricional usando la API de Edamam.	Detalle

1.3 Requisitos no funcionales

- Rendimiento: el tiempo de inicio en frío no supera 5 segundos en dispositivos con Android 8+.
- Disponibilidad offline: la app es funcional sin conexión para recetas previamente cargadas, usando caché con TTL de 30 minutos.
- Seguridad: las credenciales de Edamam se almacenan en variables de entorno de compilación (BuildConfig), no en el código fuente.
- Compatibilidad: la app es compatible con Android API 26 (Android 8.0) en adelante.
- Usabilidad: el flujo principal (Home → Región → Receta) no requiere más de 3 toques.

2. Arquitectura y Stack Tecnológico

La aplicación implementa Clean Architecture con el patrón MVVM, separando la lógica en tres capas independientes: presentación (UI + ViewModels), dominio (lógica de negocio e interfaces de repositorio) y datos (implementaciones concretas con Room y Retrofit). Hilt gestiona la inyección de dependencias entre capas.

Se eligió Android nativo con Kotlin sobre alternativas como Flutter o KMM por compatibilidad con el stack conocido del equipo y mejor integración con las APIs nativas de Android dentro del plazo disponible.

2.1 Stack tecnológico

1. UI (Interfaz de Usuario)
 - Tecnología: Jetpack Compose + Material 3
 - Justificación: UI declarativa, menos boilerplate que XML Views y con soporte nativo para animaciones.
2. Navegación
 - Tecnología: Navigation Compose
 - Justificación: Integración nativa con Compose y soporte de deep links.
3. Estado / ViewModel
 - Tecnología: StateFlow + ViewModel
 - Justificación: Reactividad unidireccional, lifecycle-aware y alineado con las guías oficiales de Android.
4. Inyección de dependencias
 - Tecnología: Hilt
 - Justificación: Generación automática de código DI con mínimo boilerplate.
5. Red
 - Tecnología: Retrofit + OkHttp
 - Justificación: Enfoque declarativo, soporte de Coroutines, interceptores para logging y manejo de errores.
6. Persistencia local
 - Tecnología: Room + DataStore
 - Justificación: Room para recetas (relacional) y DataStore para preferencias del usuario.
7. Carga de imágenes
 - Tecnología: Coil 3
 - Justificación: Ligero, nativo de Kotlin, soporte de Coroutines y caché de disco.
8. CI/CD
 - Tecnología: GitHub Actions
 - Justificación: Lint, pruebas unitarias (unit tests) y builds automáticos en cada Pull Request.

2.2 Estrategia de caché

Se implementa cache-first con TTL de 30 minutos. El repositorio verifica si existe una entrada válida en Room antes de hacer una petición de red. Si el campo `cachedAt` está dentro del TTL retorna los datos locales; si expiró o no existe, consulta la API, almacena la respuesta en Room y retorna los datos frescos. Los favoritos guardados manualmente no tienen expiración.

3. Diseño de la Interfaz

La paleta de colores está inspirada en la bandera de Colombia: rojo (#C8102E) para acentos y encabezados, amarillo (#F5A623) para badges de región, y fondo claro (#FAFAF8) para las pantallas principales. La tipografía base es sans-serif de sistema con tamaños entre 12 sp (etiquetas) y 22 sp (títulos de receta).

3.1 Pantallas de la aplicación

Pantalla	Contenido principal
1. Splash	Logo, nombre de la app, animación de carga mientras TheMealDB carga en background.
2. Home	Saludo, barra de búsqueda, tarjetas de regiones y sección de recetas populares desplazable.
3. Lista por región	Encabezado con nombre de la región, contador de recetas y lista de tarjetas con foto y nombre.
4. Detalle de receta	Foto grande, ingredientes con cantidades, instrucciones numeradas, información nutricional (Edamam) y botón de favorito.
5. Búsqueda	Resultados en tiempo real, búsquedas recientes y estado vacío cuando no hay resultados.
6. Favoritos	Lista de recetas guardadas, eliminación por deslizamiento y estado vacío cuando la lista está limpia.

4. Backlog y Planificación

4.1 Historia de usuario

ID	Como usuario	Criterio de aceptación	Prioridad
US-01	Como usuario quiero ver un splash animado al abrir la app, para tener una bienvenida visual mientras carga el contenido.	El splash aparece durante 2-3 s y luego navega automáticamente al Home.	Should
US-02	Como usuario quiero ver recetas populares en el Home, para descubrir platos sin buscar activamente.	Se muestran mínimo 5 tarjetas con foto y nombre al cargar el Home.	Must
US-03	Como usuario quiero explorar recetas por región, para descubrir la gastronomía de cada zona de Colombia.	Al tocar una tarjeta de región se lista las recetas de esa zona con un contador.	Must
US-04	Como usuario quiero ver el detalle completo de una receta, para poder prepararla correctamente.	Se muestran foto, ingredientes con cantidades e instrucciones numeradas.	Must
US-05	Como usuario quiero buscar recetas por nombre, para encontrar rápidamente un plato específico.	Los resultados se actualizan en tiempo real; si no hay resultados se muestra un mensaje.	Must
US-06	Como usuario quiero guardar recetas en favoritos, para acceder a ellas sin conexión.	Al tocar el ícono de corazón, la receta se guarda y aparece en la pantalla Favoritos.	Should
US-07	Como usuario quiero eliminar favoritos, para mantener mi lista organizada.	Al deslizar una tarjeta en Favoritos, la receta se elimina de la lista y de la BD local.	Should
US-08	Como usuario quiero leer las recetas en español, para entender los ingredientes sin barrera idiomática.	El nombre, ingredientes e instrucciones aparecen en español al cargar el detalle.	Must
US-09	Como usuario quiero ver la información nutricional, para tomar decisiones de salud informadas.	Se muestran calorías, proteínas, grasas y carbohidratos obtenidos de Edamam.	Should

4.2 Definition of Done

Una User Story se considera completada cuando:

- El código está mergeado a main mediante Pull Request con al menos 1 revisión aprobada.
- El pipeline de CI/CD (GitHub Actions) pasa sin errores: lint, unit tests y build debug.
- La funcionalidad fue verificada en un dispositivo real o emulador con API 26+.
- El ViewModel y el Repositorio de la funcionalidad tienen pruebas unitarias.

5. APIs Consumidas

5.1 TheMealDB

API principal del proyecto. Gratuita, sin autenticación requerida. Provee toda la información de recetas: fotos, ingredientes con cantidades e instrucciones de preparación. Se acceden cuatro endpoints:

1. Endpoint: **/random.php**
 - Propósito: Obtiene una receta aleatoria. Se llama múltiples veces para poblar la sección de recetas populares en el Home.
2. Endpoint: **/search.php?s={nombre}**
 - Propósito: Busca recetas por nombre. Se ejecuta con un debounce de 300 ms en la pantalla de búsqueda.
3. Endpoint: **/filter.php?a=Colombian**
 - Propósito: Lista todas las recetas clasificadas como colombianas para las pantallas de región.
4. Endpoint: **/lookup.php?i={id}**
 - Propósito: Obtiene el detalle completo de una receta, incluyendo hasta 20 ingredientes con sus cantidades e instrucciones de preparación.

5.2 Edamam Recipes API

API complementaria para información nutricional. Requiere registro gratuito en developer.edamam.com para obtener `app_id` y `app_key`, que se almacenan en variables de entorno de compilación. Su único rol es proveer calorías, proteínas, grasas y carbohidratos en la pantalla de detalle. Si la llamada falla, la sección nutricional se omite sin bloquear el resto de la receta.

5.3 Google Translate API

Se usa para traducir al español el contenido de TheMealDB, que está íntegramente en inglés. La traducción se aplica al nombre del plato, ingredientes e instrucciones. El resultado se almacena en Room para no repetir la llamada en visitas posteriores.

6. Conclusiones

Sabores de Colombia integra los contenidos del curso en una sola solución funcional: arquitectura limpia (Clean Architecture + MVVM), consumo de servicios REST con Retrofit, persistencia local con caché TTL en Room, inyección de dependencias con Hilt e interfaz declarativa con Jetpack Compose.

El problema abordado es real: la barrera idiomática y la dispersión de información sobre gastronomía colombiana en plataformas digitales. La solución centraliza el contenido en español, organizado por región y accesible sin conexión para recetas ya visitadas.

Como trabajo futuro se identifica la migración a Kotlin Multiplatform Mobile para soporte iOS, la implementación de autenticación con Firebase para sincronizar favoritos entre dispositivos, y la construcción de una base de datos propia de recetas organizadas por las cinco regiones geográficas de Colombia, superando la limitación de TheMealDB que no distingue entre regiones internas del país.

Referencias

Android Developers. (2024). Guide to app architecture. Google LLC. <https://developer.android.com/topic/architecture>

Android Developers. (2024). Room Persistence Library. Google LLC. <https://developer.android.com/training/data-storage/room>

Android Developers. (2024). Hilt for dependency injection. Google LLC. <https://developer.android.com/training/dependency-injection/hilt-android>

Edamam. (2024). Recipe Search API v2. <https://developer.edamam.com/edamam-recipe-api>

TheMealDB. (2024). TheMealDB API v1. <https://www.themealdb.com/api.php>

Universidad de Santander — UDES. (2026). Unidad 12: Proyecto Integrador — Guía de contenido. Aplicaciones Móviles.